Overview of Clearing of the EAC Market

1 Clearing of the Market Basics & background

1.1 Models

In order to clear the market, we define three main components: **Buy Orders**, **Sell Orders**, and **Baskets**.

Buy Orders

A Buy Order represents what NESO wants to purchase. Each buy order includes:

- **ID**: a unique identifier.
- **Product**: e.g. DRH, NQR, DML.
- **Price per MW**.
- **Volume**: how much NESO wishes to buy.
- **Substitutional Family**: a grouping representing substitutable buy orders.
- **Paradoxical Acceptance Flag**: whether this buy order may be "accepted paradoxically" (i.e. accepted even if its own welfare becomes negative).
- **Minimum Acceptance Ratio**.

Sell Orders

A Sell Order represents what market participants want to sell. Each sell order includes:

- **ID**: unique identifier.
- **Basket**: the basket the sell order belongs to.
- **Quantities**: how much of each product is being sold.
- **Price per MW**.

- **Type**: one of *parent*, *child*, or *substitutable_child*.
- **Minimum Acceptance Ratio**.

Baskets

A basket groups together sell orders. Accepting a sell order implies accepting its basket. Each basket contains:

- **Basket ID**: unique identifier.
- **Unit**: the physical unit (e.g. a battery).
- **Concomitant**: a list of baskets that cannot be accepted simultaneously.
- **Looped To**: baskets that must be accepted together.

1.2 Validators

The `validators.py` file contains two core functionalities:

Loop Family Construction

Some baskets must be accepted together. Using Breadth-First Search, we identify all such connected baskets and return loop families.

Unit Capacity Validation

We ensure accepted sell orders do not exceed their associated unit's physical capacity (e.g. a battery with a 100 MW limit).

1.2.1 Tests

The test cases for validators are written logically and with code commenting for ease of following.

1.3 Solver

The `solver.py` file defines a thin wrapper around the CBC (COIN-OR Branch & Cut) solver. CBC is well-suited for linear and mixed-integer problems and integrates seamlessly with PuLP.

1.4 Rounding Procedures

The `rounding.py` file handles two crucial steps:

1. Price Rounding (EAC Section 8 Rule)

For each product, the unrounded market clearing price is rounded **upwards to the nearest penny**. This ensures no participant is paid below the MCP after rounding.

2. Rounding and Residual Distribution

Sell-Side Rounding Given acceptance ratios x_s:

(a) Compute unrounded acceptance:

$$\text{accepted\_unrounded} = (\text{total quantity}) \times x_s.$$

(b) Apply EAC rules:

- Parent & child: round to nearest integer.
- Substitutable child: round down.

(c) Distribute rounded sell volume across products proportionally, using largest-remainder tie-breaking.

Buy-Side Rounding Buy order acceptance is rounded to the nearest integer:

$$\text{accepted\_buy\_rounded} = \text{round}(V_b x_b).$$

Residual Correction For each product:

$$\text{residual} = \text{rounded\_buys} - \text{rounded\_sells}.$$

- If residual > 0: decrement buyer allocations (too many buys).
- If residual < 0: increment buyer allocations (too few buys).

Only **buyer volumes** absorb rounding error, per EAC rules.
Adjustment order is deterministic:

- When increasing buys: lowest price first.
- When decreasing buys: highest price first.

1.4.1 Tests

The test cases for rounding are written logically and with code commenting for ease of following.

2 Volume MILP Formulation

Sets

$\mathcal{P}$: set of products,

$\mathcal{B}$: buy orders (including overholding pseudo-buys. This is where the buy order can accept more volume than

$\mathcal{S}$: sell orders,

$\mathcal{U}$: baskets.

Parameters

c_b : buy order price, V_b : buy order volume,

c_s : sell order price, $q_{s,p}$: quantity of product p from sell order s.

Decision Variables

$x_b \in [0,1]$	acceptance ratio of buy order b,
$x_s \in [0,1]$	acceptance ratio of sell order s (binary if parent),
$y_u \in \{0,1\}$	basket acceptance.

Objective: Maximise Welfare

$$\max \left(\sum_{b \in \mathcal{B}} c_b V_b x_b \; - \; \sum_{s \in \mathcal{S}} c_s \left(\sum_{p \in \mathcal{P}} q_{s,p} \right) x_s \right). \tag{1}$$

We have to follow this formula for all the time windows.

Constraints

1. Parent acceptance (if we accept the parent then we have accepted the basket buy order):
$$x_s = y_{basket(s)} \quad \forall s \text{ parent.}$$

2. Child acceptance (The child acceptance must be less than the binary acceptance of the parent sell order):
$$x_s \leq y_{basket(s)} \quad \forall s \text{ child or substitutable child.}$$

3. Substitutable exclusivity (The summation of all the sell orders for the subsitutionable childs wihtin a basket must be less than or equal to 1):
$$\sum_{s \in \text{subs}(u)} x_s \leq 1 \quad \forall u \in \mathcal{U}.$$

4. Concomitant restriction (This means we cannot fully accept two baskets):

$$y_u + y_v \leq 1 \quad \forall (u, v) \text{ concomitant.}$$

5. Loop family consistency:

$$y_u = y_v \quad \forall u, v \text{ in same loop family.}$$

6. Product balance (the amount of product we buy and sell must be equal):

$$\sum_s q_{s,p} x_s = \sum_{b: product(b)=p} V_b x_b \quad \forall p.$$

7. Buy-side substitutability (We cannot fully accept two different buy orders which are in teh same subsitutionable family):

$$\sum_{b \in \text{buy\_subs\_family}_i} x_b \leq 1.$$

3 Pricing Linear Program (EAC)

Decision Variables

$$P_p \in [P_{\min}, P_{\max}] \quad \forall p.$$

Objective

$$\min_P \sum_{s \in \mathcal{S}} x_s^{\text{fixed}} \sum_{p \in \mathcal{P}} q_{s,p} P_p.$$

Constraints

1. **Sell-level non-negative profit.**
   This constraint ensures that every accepted sell order does not make a loss at the clearing price.

$$\sum_p q_{s,p} P_p \geq c_s \sum_p q_{s,p}$$

2. **Basket-level non-negative profit.**
   A basket may contain multiple sell orders belonging to the same physical unit. Since these sell orders are technically linked, the EAC requires that *the aggregate of all accepted sells in a basket must also be profitable.* The term

$$\sum_{s \in \mathcal{S}_u} x_s^{\text{fixed}} \left(\sum_p q_{s,p} P_p - c_s \sum_p q_{s,p} \right)$$

computes the *total net profit* of all accepted sell orders within basket u (each term is revenue minus cost, weighted by the acceptance ratio). The constraint

$$\sum_{s \in \mathcal{S}_u} x_s^{\text{fixed}} \left(\sum_p q_{s,p} P_p - c_s \sum_p q_{s,p} \right) \geq 0$$

ensures that, even if individual sells are profitable, their *combined effect* is also non-negative. This captures the idea that baskets represent physical assets whose bids must be profitable as a whole.

3. **Loop-family non-negative profit.**
Some baskets are linked into *loop families* through "looped_to" relationships, meaning they must be accepted together. Because these baskets are jointly binding, the EAC requires that *the entire loop family must be profitable as a group*. For a loop family F, the total net profit is:

$$\sum_{s \in \bigcup_{u \in F} \mathcal{S}_u} x_s^{\text{fixed}} \left(\sum_p q_{s,p} P_p - c_s \sum_p q_{s,p} \right).$$

The constraint

$$\sum_{s \in \bigcup_{u \in F} \mathcal{S}_u} x_s^{\text{fixed}} \left(\sum_p q_{s,p} P_p - c_s \sum_p q_{s,p} \right) \geq 0$$

guarantees that *the entire collection of baskets in the loop family* is profitable at the clearing prices. This prevents any situation where a group of technically linked baskets is accepted even though their combined revenue is lower than their combined bid cost.